CyberAudit

PDF Report Generation

WeasyPrint + Celery async task - Server-side score/grade - /media/reports/ storage

Project	CyberAudit Platform
Module	apps/reports/
Model	AuditReport (UUID PK, OneToOne -> AuditRequest)
7 grades	A / B+ / B / C / D / E / F (threshold-based, server-side)
4 severity weights	critical=25 / high=10 / medium=4 / low=1 (deducted from 100)
Anti-tampering	score+grade ALWAYS computed server-side -- client values ignored
PDF engine	WeasyPrint HTML -> PDF, stored in MEDIA_ROOT/reports/
Celery task	generate_pdf_task(report_id) -- autoretry x3, backoff
3 endpoints	POST generate-report/ | GET report/ (PDF) | GET report/data/ (JSON)
Anti-enumeration	Non-owner returns 404 (not 403) to hide report existence
Tests	9 tests across 4 classes
Date	June 2026 / Juin 2026

1) AuditReport Model

1) Modele AuditReport

AuditReport has a OneToOneField to AuditRequest, one audit, one report. The grade and security_score are never set from client input: they are computed exclusively by services.py from the findings array. pdf_path stores the filesystem path after Celery generates it.

AuditReport a un OneToOneField vers AuditRequest, un audit, un rapport. Le grade et security_score ne sont jamais définis depuis le client : ils sont calculés exclusivement par services.py à partir du tableau findings. pdf_path stocke le chemin du fichier après la génération par Celery.

```
class AuditReport(models.Model):
    class Grade(models.TextChoices):
        A     = 'A',  'A -- Excellent'
        B_PLUS = 'B+', 'B+ -- Tres bon'
        B     = 'B',  'B -- Bon'
        C     = 'C',  'C -- Moyen'
        D     = 'D',  'D -- Faible'
        E     = 'E',  'E -- Mauvais'
        F     = 'F',  'F -- Critique'

    id            = models.UUIDField(primary_key=True, default=uuid.uuid4,
editable=False)
    audit_request = models.OneToOneField('audits.AuditRequest', on_delete=CASCADE,
                                         related_name='report')
    summary        = models.TextField(blank=True)
    verdict        = models.CharField(max_length=255, blank=True)
    grade          = models.CharField(max_length=2, choices=Grade.choices,
default=Grade.C)
    security_score = models.PositiveSmallIntegerField(default=50)  # 0-100
    findings       = models.JSONField(default=list, blank=True)
    pdf_path       = models.CharField(max_length=255, blank=True)
    generated_at   = models.DateTimeField(null=True, blank=True)
    created_at     = models.DateTimeField(auto_now_add=True)
    updated_at     = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'audit_reports'
        ordering = ['-created_at']
```

Field reference / Référence des champs

Field	Type	Purpose EN / FR
id	UUIDField (PK)	UUID v4 auto generated. Safe for public API exposure. UUID v4 généré automatiquement. Sécurisé pour l'exposition API publique.
audit_request	OneToOneField	1:1 link. related_name='report' allows audit.report shortcut. Lien 1:1. related_name='report' permet le raccourci audit.report.
grade	CharField (choices)	A / B+ / B / C / D / E / F. ALWAYS computed server-side. Default C. TOUJOURS calcule cote serveur. Defaut C.
security_score	PositiveSmallInt	0-100 integer. ALWAYS computed server-side. Default 50. 0-100. TOUJOURS calcule cote serveur. Defaut 50.
findings	JSONField (list)	List of sanitized vulnerability objects. Whitelist-filtered. Liste d'objets de vulnerabilite nettoyes. Filtrage par whitelist.
pdf_path	CharField(255)	Empty until Celery generates the file. Absolute filesystem path. Vide jusqu'à la génération par Celery. Chemin absolu du fichier.
generated_at	DateTimeField (nullable)	NULL until PDF written. Set by Celery task on success. NULL jusqu'a l'ecriture du PDF. Defini par la tache Celery.

2) Score & Grade Engine : services.py

2) Moteur de score et grade : services.py

services.py is the single source of truth for scoring. All functions are pure (no DB calls). The score starts at 100 and deducts penalty points per finding severity. The grade is a threshold lookup on the final score. The client can never influence score or grade.

services.py est la source unique de vérité pour le scoring. Toutes les fonctions sont pures (pas d'appels DB). Le score part de 100 et déduit des pénalités selon la sévérité de chaque finding. Le grade est un lookup de seuil sur le score final. Le client ne peut jamais influencer le score ou le grade.

Severity weights & Grade thresholds

Severity	Penalty	Score range	Grade
critical	-25 pts	>= 90	A
high	-10 pts	>= 80	B+
medium	-4 pts	>= 70	B
low	-1 pt	>= 55	C (default)
(none)	0 pts	>= 40	D
		>= 20	E
		0-19	F

services.py core functions

```
# Score starts at 100 and deducts per finding
def compute_score(findings):
    if not findings:
        return 100  # no findings = perfect score
    deduction = sum(SEVERITY_WEIGHTS.get(f.get('severity','Low').lower(), 1)
                    for f in findings)
    return max(0, min(100, 100 - deduction))  # clamp 0-100

# Grade = first threshold score meets
def score_to_grade(score):
    for threshold, grade in GRADE_THRESHOLDS:  # ordered 90 -> 80 -> ... -> 0
        if score >= threshold:
            return grade
    return 'F'

# Whitelist-filter + normalize severity
def sanitize_findings(raw_findings):
    if not isinstance(raw_findings, list):
        return []
    out = []
    for f in raw_findings:
        if not isinstance(f, dict):
            continue
        clean = {k: str(f.get(k, '')).strip() for k in ALLOWED_FINDING_FIELDS}
        clean['severity'] = normalize_severity(clean['severity'])
        out.append(clean)
    return out

# ALLOWED_FINDING_FIELDS = ('severity', 'asset', 'description', 'recommendation')
```

Score examples / Exemples de score

Findings	Deduction	Score	Grade
(none)	0	100	A
1 critical	-25	75	B
2 high + 1 medium	-24	76	B
3 critical	-75	25	E
5 critical	-125 (capped)	0	F

3) Celery Task (generate_pdf_task)

3) Tache Celery (generate_pdf_task)

generate_pdf_task is a 5-step pipeline: render HTML template, convert to PDF with WeasyPrint, save to MEDIA_ROOT/reports/, update the model, then send a report_ready notification to the client. All steps inside the task ensure atomicity via retry on any exception.

generate_pdf_task est un pipeline en 5 étapes : rendu du template HTML, conversion en PDF avec WeasyPrint, sauvegarde dans MEDIA_ROOT/reports/, mise à jour du modèle, puis envoi d'une notification report_ready au client. Toutes les étapes sont dans la tâche, la rendant rentable sur toute exception.

Task pipeline

```
@shared_task(bind=True, autoretry_for=(Exception,), max_retries=3,
             default_retry_delay=10, retry_backoff=True, retry_backoff_max=600)
def generate_pdf_task(self, report_id):
    from weasyprint import HTML  # late import -- avoids circular imports
    from apps.notifications.services import create_and_send
    from .models import AuditReport

    # Step 1: fetch report with related objects (1 query)
    report = AuditReport.objects.select_related(
        'audit_request__client', 'audit_request__pack'
    ).get(pk=report_id)

    # Step 2: render Django HTML template
    html_string = render_to_string('reports/report.html', {
        'report': report,
        'audit':  report.audit_request,
        'client': report.audit_request.client,
        'pack':   report.audit_request.pack,
    })

    # Step 3: WeasyPrint HTML -> PDF
    media_root = Path(settings.MEDIA_ROOT) / 'reports'
    media_root.mkdir(parents=True, exist_ok=True)  # create dir if missing
    pdf_filename = f'{report.audit_request.reference}.pdf'
    pdf_path = media_root / pdf_filename
    HTML(string=html_string).write_pdf(str(pdf_path))

    # Step 4: persist pdf_path + generated_at
    report.pdf_path    = str(pdf_path)
    report.generated_at = timezone.now()
    report.save(update_fields=['pdf_path', 'generated_at'])

    # Step 5: notify client -- report_ready
    create_and_send(
        user=report.audit_request.client,
        request=report.audit_request,
        type_=Notification.Type.REPORT_READY,
        subject=f'Your audit report is ready -- {report.audit_request.reference}',
        message=f'Grade {report.grade}, Score {report.security_score}/100.',
    )
    return f'PDF genere : {pdf_path}'
```

Late imports	weasyprint, create_and_send, AuditReport imported inside function body. Prevents circular import at module load and allows easy mocking in tests. Imports dans le corps. Evite les imports circulaires et facilite le mock en tests.
select_related x2	Fetches audit_request + client + pack in 1 SQL JOIN. No N+1. Récupère audit_request + client + pack en 1 JOIN SQL. Pas de N+1.
mkdir(parents, exist_ok)	Create MEDIA_ROOT/reports/ if missing. Safe for first run. Créer MEDIA_ROOT/reports/ si absent. Sécurisé pour le premier lancement.
update_fields	Only pdf_path + generated_at written. No full-row UPDATE. Uniquement pdf_path + generated_at ecrits.
Retry on any Exception	Any step (template render, WeasyPrint, file write, notification) triggers retry x3 with backoff. N'importe quelle étape (template, WeasyPrint, fichier, notification) déclenche retry x3.

4) GenerateReportView : POST /generate-report/

4) GenerateReportView : POST /generate-report/

Admin-only endpoint. The key security invariant: security_score and grade sent by the client in the POST body are silently ignored. They are always recalculated server-side from the sanitized findings. The view uses update_or_create so re-generating a report overwrites the previous one.

Endpoint admin uniquement. L'invariant de sécurité cle : security_score et grade envoyés par le client dans le corps POST sont silencieusement ignores. Ils sont toujours recalculés cote serveur à partir des findings nettoyés. La vue utilise update_or_create donc régénérer un rapport écrase le précèdent.

```
class GenerateReportView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request, audit_id):
        if request.user.role != 'admin':
            raise PermissionDenied('Only an administrator can generate a report.')

        audit = _get_audit_or_404(audit_id)

        # ANTI-TAMPERING: ignore client-sent score/grade
        findings = sanitize_findings(request.data.get('findings', []))
        security_score, grade = compute_score_and_grade(findings)  # server-side

        # Upsert: max 1 report per AuditRequest
        report, _ = AuditReport.objects.update_or_create(
            audit_request=audit,
            defaults={
                'summary': str(request.data.get('summary', '')).strip(),
                'verdict': str(request.data.get('verdict', '')).strip(),
                'security_score': security_score,  # server value
                'grade': grade,                    # server value
                'findings': findings,
                'pdf_path': '',                    # reset -> regenerate
                'generated_at': None,
            },
        )

        generate_pdf_task.delay(str(report.id))  # async

        return Response({
            'id': str(report.id),
            'status': 'generating',
            'security_score': security_score,
            'grade': grade,
            'findings_count': len(findings),
        }, status=HTTP_202_ACCEPTED)
```

role != 'admin' -> 403	PermissionDenied raised before any DB read. Client role blocked immediately. PermissionDenied leve avant toute lecture DB. Rôle client bloque immédiatement.
sanitize_findings()	Whitelist: only severity/asset/description/recommendation kept. All other keys stripped. Whitelist : uniquement ces 4 clés conservées. Toutes les autres supprimees.
compute_score_and_grade()	Server calculates score and grade from sanitized findings. Client value in POST body ignored. Le serveur calcule score et grade. La valeur du client dans le POST est ignorée.
update_or_create	OneToOneField enforces 1 report per audit. Re-calling resets pdf_path+generated_at. OneToOneField impose 1 rapport par audit. Rappel reinitialise pdf_path+generated_at.
202 Accepted	PDF generation is async. Immediate response with computed score/grade + task queued. Generation PDF asynchrone. Réponse immédiate avec score/grade + tache en file.

5) Download & Data Views + URL Routing

5) Vues de telechargement et JSON + Routage URL

ReportDownloadView streams the PDF as FileResponse with strict Cache-Control. ReportDataView returns the JSON serialization. Both apply the same anti-enumeration guard: non-owner gets 404, not 403, to avoid revealing whether a report exists.

ReportDownloadView diffuse le PDF en FileResponse avec un Cache-Control strict. ReportDataView retourne la sérialisation JSON. Les deux appliquent la même protection anti-énumération : non-propriétaire reçoit 404 et non 403, pour ne pas révéler l'existence d'un rapport.

ReportDownloadView : GET /report/

```
class ReportDownloadView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request, audit_id):
        audit = _get_audit_or_404(audit_id)
        # Anti-enumeration: non-owner AND non-admin -> 404 (not 403)
        if request.user.role != 'admin' and audit.client_id != request.user.id:
            raise Http404 from None

        try:
            report = audit.report  # OneToOne reverse accessor
        except AuditReport.DoesNotExist:
            raise NotFound('No report for this request.') from None

        if not report.pdf_path:
            return Response({'detail': 'PDF generation in progress.'},
                            status=HTTP_202_ACCEPTED)

        try:
            f = open(report.pdf_path, 'rb')
        except FileNotFoundError:
            raise NotFound('PDF file not found on disk.') from None

        response = FileResponse(f, content_type='application/pdf')
        response['Content-Disposition'] = f'attachment;
filename="{audit.reference}.pdf"'
        response['Cache-Control'] = 'no-store, private'
        return response
```

ReportDataView + URL routing

```python
class ReportDataView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request, audit_id):
        audit = _get_audit_or_404(audit_id)
        if request.user.role != 'admin' and audit.client_id != request.user.id:
            raise Http404 from None
        try:
            report = audit.report
        except AuditReport.DoesNotExist:
            raise NotFound('No report for this request.') from None
        return Response(AuditReportSerializer(report).data)

# reports/urls.py -- nested under /api/
urlpatterns = [
    path('audits/<uuid:audit_id>/generate-report/', GenerateReportView.as_view()),
    path('audits/<uuid:audit_id>/report/',          ReportDownloadView.as_view()),
    path('audits/<uuid:audit_id>/report/data/',     ReportDataView.as_view()),
]
```

URL	Method	Auth	Response	Description EN / FR
POST /api/audits/{id}/generate-report/	POST	Admin	202	Sanitize + score + upsert + Celery dispatch. Client score ignored. Nettoyage + score + upsert + envoi Celery.
GET /api/audits/{id}/report/	GET	Owner/Admin	200 / 202 / 404	Stream PDF (200) or 202 if still generating or 404 if missing. PDF en stream (200), 202 si génération en cours, 404 si absent.
GET /api/audits/{id}/report/data/	GET	Owner/Admin	200 / 404	JSON report data via AuditReportSerializer. Données JSON du rapport via AuditReportSerializer.

6) AuditReportSerializer + Response Shapes

6) AuditReportSerializer + Formats de réponse

```
class AuditReportSerializer(serializers.ModelSerializer):
    audit_reference = serializers.CharField(source='audit_request.reference',
read_only=True)

    class Meta:
        model = AuditReport
        fields = ['id','audit_reference','summary','verdict','grade',
                  'security_score','findings','pdf_path','generated_at']
        read_only_fields = fields
```

POST /generate-report/ (202 Accepted body)

```
// Immediate response (PDF not yet ready):
{ 'id': '3fa85f64-...', 'status': 'generating', 'security_score': 75, 'grade':
'B', 'findings_count': 3 }
```

GET /report/data/ (200 OK body)

```
{ 'id': '3fa85f64-...',
  'audit_reference': 'DOSSIER-2026-0042',
  'summary': 'Infrastructure audit revealed 3 vulnerabilities.',
  'verdict': 'Immediate patching recommended.',
  'grade': 'B',
  'security_score': 75,
  'findings': [
    { 'severity': 'Critical', 'asset': 'VPN gateway',
      'description': 'OpenVPN outdated', 'recommendation': 'Update to 2.6.x' },
    { 'severity': 'High', 'asset': 'Admin panel',
      'description': 'No MFA', 'recommendation': 'Enable TOTP' }
  ],
  'pdf_path': '/media/reports/DOSSIER-2026-0042.pdf',
  'generated_at': '2026-06-17T14:35:00Z'
}
```

GET /report/ (202 if still generating)

```
// pdf_path still empty:
{ 'detail': 'PDF generation in progress.' }  // 202 Accepted
```

7) Tests : 9 tests across 4 classes

7) Tests : 9 tests en 4 classes

Tests cover model defaults, all 3 endpoint RBAC scenarios (admin/client/anonymous), anti-enumeration (other client gets 404), the PDF-not-ready 202 path, and the Celery task with WeasyPrint mocked.

Les tests couvrent les valeurs par defaut du modèle, les 3 scenarios RBAC pour les endpoints (admin/client/anonyme), l'anti-énumération (autre client reçoit 404), le chemin 202 PDF-pas-prêt, et la tache Celery avec WeasyPrint mocke.

Class	Test	HTTP	Assertion EN / FR
TestAuditReportModel	test_default_grade_and_score	--	grade=='C', security_score==50, pdf_path==''. grade=='C', security_score==50, pdf_path==''.
TestAuditReportModel	test_str_contains_reference_and_grade	--	str() contains grade 'A' and audit.reference. str() contient le grade et la reference.
TestGenerateReportEndpoint	test_admin_can_trigger	202	Admin: 202, report row created, mock_delay called once. Admin: 202, ligne rapport creee, mock_delay appele une fois.
TestGenerateReportEndpoint	test_client_cannot_trigger	403	Client role -> 403 PermissionDenied. Role client -> 403. PermissionDenied.
TestGenerateReportEndpoint	test_anonymous_returns_401	401	No token -> 401 Unauthorized. Sans token -> 401.
TestReportDownloadEndpoint	test_owner_can_download	200	Content-Type: application/pdf. Cache-Control: no-store, private. Content-Type PDF.

			Cache-Control no-store.
TestReportDownloadEndpoint	test_other_client_gets_404	404	Anti-enumeration. Client B on Client A's audit -> 404 (not 403). Anti-énumération. Client B sur l'audit de Client A -> 404 (pas 403).
TestReportDownloadEndpoint	test_pdf_not_generated_returns_202	202	Report exists, pdf_path empty -> 202 'generating'. Rapport présent, pdf_path vide -> 202.
TestGeneratePdfTask	test_task_creates_pdf_and_notifies	--	@patch WeasyPrint. After task: pdf_path set, generated_at set, report_ready notification created. @patch WeasyPrint. Apres tache: pdf_path defini, notification report_ready crée.

8. Security & Design Summary

8. Recapitulative sécurité et conception

Feature	Detail EN / FR
Anti-tampering	Client-sent security_score and grade SILENTLY IGNORED. Always recomputed from findings. Score et grade envoyés par le client SILENCIEUSEMENT IGNORES. Toujours recalcules depuis les findings.
Findings whitelist	sanitize_findings() keeps only 4 fields: severity/asset/description/recommendation. All extra keys stripped. sanitize_findings() ne conserve que 4 champs. Toutes les clés supplémentaires supprimées.
Admin-only generate	role != 'admin' raises PermissionDenied (403) before any DB access. role != 'admin' leve PermissionDenied avant tout accès DB.
Anti-enumeration	Non-owner returns 404 (not 403). Reveals neither report existence nor ownership. Non-propriétaire reçoit 404. Ne révèle ni l'existence ni la propriété.
PDF Cache-Control	no-store, private : browser/proxy must not cache PDF. no-store, private : le navigateur/proxy ne doit pas mettre en cache.
Content-Disposition	attachment filename='{reference}.pdf' : forces download, sets safe filename. attachment filename='{reference}.pdf' : force le téléchargement, nom de fichier sécurise.
Async PDF generation	202 Accepted + Celery task. API response not blocked by WeasyPrint latency. 202 Accepté + tache Celery. Réponse API non bloquée par la latence WeasyPrint.
update_or_create	OneToOneField enforces max 1 report. Re-generate resets pdf_path+generated_at cleanly. OneToOneField impose max 1 rapport. Régénérer réinitialise proprement.
Server-side score	Single source of truth in services.py. Pure functions, testable without Django. Source unique de vérité dans services.py. Fonctions pures testables sans Django.
WeasyPrint mock	@patch('weasyprint.HTML') in tests. No actual PDF rendered in CI, fast and side-effect-free. @patch('weasyprint.HTML') en tests. Pas de vrai PDF en CI, rapide et sans effets de bord.